

JavaScript Array Methods for React

Overview

React development-এর জন্য JavaScript Array-এর সব method সমানভাবে গুরুত্বপূর্ণ নয়। বাস্তব React project-এ প্রায় **90% array-related কাজ** করতে নিচের methods ভালোভাবে জানলেই শক্ত foundation তৈরি হয়:

```
1. map()
2. filter()
3. find()
4. includes()
5. some()
6. reduce()
7. sort()
```

Core Focus: `map()`, `filter()`, `find()`, `includes()`, `some()`, `reduce()`, `sort()`

These methods are commonly used for rendering API data, searching, filtering, checking conditions, calculating totals, and sorting UI data.

1. `map()`

Definition

`map()` একটি array-এর প্রতিটি element-এর উপর একটি function চালিয়ে **একটি নতুন array** তৈরি করে।

React-এ এটি সবচেয়ে বেশি ব্যবহার করা হয় **list**, **card**, **table row**, **menu** এবং **component render করার জন্য**।

Syntax

```
const newArray = array.map((item, index) => {
  return transformedValue;
});
```

Basic Example

```
const numbers = [1, 2, 3, 4];

const doubled = numbers.map(number => number * 2);

console.log(doubled);
```

Output:

```
[2, 4, 6, 8]
```

React Example

```
const products = [
  { id: 1, name: "Laptop" },
  { id: 2, name: "Mouse" },
  { id: 3, name: "Keyboard" }
];

function ProductList() {
  return (
    <div>
      {products.map(product => (
        <div key={product.id}>
          <h3>{product.name}</h3>
        </div>
      ))}
    </div>
  );
}
```

Remember

map() = প্রতিটি item নিয়ে কাজ করে + নতুন array দেয়

2. filter()

Definition

filter() একটি condition অনুযায়ী array থেকে matching elements নিয়ে **একটি নতুন array** তৈরি করে।

React-এ search, category filter, active/inactive filter, price range এবং status filtering-এর জন্য খুব গুরুত্বপূর্ণ।

Syntax

```
const filteredArray = array.filter((item, index) => {
  return condition;
});
```

Basic Example

```
const ages = [15, 18, 21, 25, 30];

const adults = ages.filter(age => age >= 18);

console.log(adults);
```

Output:

```
[18, 21, 25, 30]
```

React Example

```
const activeUsers = users.filter(user => user.isActive);

{activeUsers.map(user => (
  <p key={user.id}>{user.name}</p>
))}
```

Remember

```
filter() = condition মিলে এমন একাধিক item বের করে
```

3. find()

Definition

`find()` array-এর মধ্যে condition পূরণ করা **প্রথম element** return করে।

কোনো নির্দিষ্ট user, product, order বা item খুঁজতে এটি খুব useful।

Syntax

```
const result = array.find((item, index) => {  
  return condition;  
});
```

কোনো match না পেলে `undefined` return করে।

Basic Example

```
const users = [  
  { id: 1, name: "Rahim" },  
  { id: 2, name: "Karim" },  
  { id: 3, name: "Hasan" }  
];  
  
const user = users.find(user => user.id === 2);  
  
console.log(user);
```

Output:

```
{ id: 2, name: "Karim" }
```

React Example

```
const product = products.find(product => product.id === selectedId);  
  
return (  
  <div>  
    <h2>{product?.name}</h2>  
    <p>{product?.price}</p>  
  </div>  
);
```

Remember

`find()` = প্রথম matching item বের করে

4. `includes()`

Definition

`includes()` array-এর মধ্যে কোনো নির্দিষ্ট value আছে কিনা check করে।

এটি `true` অথবা `false` return করে।

Syntax

```
const result = array.includes(value);
```

Basic Example

```
const skills = ["HTML", "CSS", "JavaScript"];  
  
console.log(skills.includes("JavaScript"));
```

Output:

```
true
```

React Example

```
const categories = ["mobile", "laptop", "tablet"];  
  
const showMobile = categories.includes("mobile");  
  
return showMobile && <MobileProducts />;
```

Remember

```
includes() = value আছে? → true / false
```

`includes()` primitive value check-এর জন্য convenient। Object-এর ক্ষেত্রে reference equality-এর কারণে সাধারণত `find()` বা `some()` বেশি useful।

5. `some()`

Definition

`some()` check করে array-এর **কমপক্ষে একটি element** condition পূরণ করছে কিনা। একটি match পেলেই `true` return করে।

Syntax

```
const result = array.some((item, index) => {  
  return condition;  
});
```

Basic Example

```
const prices = [500, 1200, 3000];  
  
const hasExpensiveProduct = prices.some(price => price > 2000);  
  
console.log(hasExpensiveProduct);
```

Output:

```
true
```

React Example

```
const hasOutOfStock = products.some(product => product.stock === 0);

return (
  <div>
    {hasOutOfStock && (
      <p>Some products are currently out of stock.</p>
    )}
  </div>
);
```

Remember

`some()` = অন্তত ১টা match করলেই true

6. `reduce()`

Definition

`reduce()` array-এর সব elements process করে **একটি final result** তৈরি করে।

React application-এ total price, quantity, score, summary, statistics এবং aggregated data তৈরি করতে এটি খুব useful।

Syntax

```
const result = array.reduce((accumulator, currentValue) => {
  return updatedAccumulator;
}, initialValue);
```

Basic Example

```
const prices = [500, 1000, 1500];

const total = prices.reduce((sum, price) => {
  return sum + price;
}, 0);

console.log(total);
```

Output:

```
3000
```

React Example

```
const cart = [
  { id: 1, name: "Laptop", price: 70000, quantity: 1 },
  { id: 2, name: "Mouse", price: 1000, quantity: 2 }
];

const total = cart.reduce(
  (sum, item) => sum + item.price * item.quantity,
  0
);

return <h2>Total:  $\text{\`{total}}$ </h2>;
```

Remember

```
reduce() = অনেকগুলো value → একটি final result
```

7. `sort()`

Definition

`sort()` array-এর elements-কে একটি নির্দিষ্ট order অনুযায়ী সাজায়।

React-এ price, name, rating, date বা score অনুযায়ী data sorting করতে এটি খুব useful।

Syntax

```
array.sort((a, b) => {
  return comparisonResult;
});
```

Number Ascending

```
const numbers = [50, 10, 30, 20];

numbers.sort((a, b) => a - b);

console.log(numbers);
```

Output:

```
[10, 20, 30, 50]
```

React Example

```
const sortedProducts = [...products].sort(
  (a, b) => a.price - b.price
);

return (
  <div>
    {sortedProducts.map(product => (
      <p key={product.id}>
        {product.name} - ট{product.price}
      </p>
    ))}
  </div>
);
```

Important

`sort()` original array modify করে। React state-এর ক্ষেত্রে সরাসরি state array sort করা avoid করা উচিত।

Use:

```
const sortedProducts = [...products].sort(
  (a, b) => a.price - b.price
);
```

Not:

```
products.sort((a, b) => a.price - b.price);
```

Remember

`sort()` = array-এর order পরিবর্তন করে

React-এ সবচেয়ে বেশি ব্যবহার হওয়া Combination

Real project-এ একটি method-এর চেয়ে **multiple methods** একসাথে বেশি ব্যবহার হবে।

Common pattern:

```
filter() → data filter  
  ↓  
sort()   → data sort  
  ↓  
map()    → UI render
```

আর নির্দিষ্ট item-এর জন্য:

```
find()    → single item  
includes() → simple value check  
some()    → condition check  
reduce()  → total / summary
```

Real-World React Project 1: Product Search + Category Filter

এই example-এ `filter()`, `map()` এবং `includes()` একসাথে ব্যবহার করা হয়েছে।

Features

```
Search products  
Category filtering  
Render product cards
```

```
import { useState } from "react";

const products = [
  {
    id: 1,
    name: "HP Laptop",
    category: "laptop",
    price: 65000
  },
  {
    id: 2,
    name: "Samsung Phone",
    category: "mobile",
    price: 25000
  },
  {
    id: 3,
    name: "Gaming Laptop",
    category: "laptop",
    price: 95000
  },
  {
    id: 4,
    name: "Redmi Phone",
    category: "mobile",
    price: 18000
  }
];

function ProductSearch() {
  const [search, setSearch] = useState("");
  const [category, setCategory] = useState("all");

  const filteredProducts = products.filter(product => {
    const matchesSearch = product.name
      .toLowerCase()
      .includes(search.toLowerCase());

    const matchesCategory =
      category === "all" ||
      [product.category].includes(category);

    return matchesSearch && matchesCategory;
  });

  return (
    <div>
      <input
        type="text"
        placeholder="Search product..."
        value={search}
        onChange={e => setSearch(e.target.value)}
      />

      <select
        value={category}
        onChange={e => setCategory(e.target.value)}
      />
    </div>
  );
}
```

```

    >
    <option value="all">All</option>
    <option value="laptop">Laptop</option>
    <option value="mobile">Mobile</option>
  </select>

  <div>
    {filteredProducts.map(product => (
      <div key={product.id}>
        <h3>{product.name}</h3>
        <p>Category: {product.category}</p>
        <p>Price: ₹{product.price}</p>
      </div>
    ))}
  </div>
</div>
);
}

export default ProductSearch;

```

Methods Used

```

filter() → matching products বের করা
includes() → category/value check
map() → products UI-তে render করা

```

Real-World React Project 2: Shopping Cart

এই project-এ `find()`, `reduce()`, `map()` এবং `some()` ব্যবহার করা হয়েছে।

Features

```

Add products
Show cart items
Calculate total
Detect out-of-stock items

```

```
import { useState } from "react";

const initialProducts = [
  {
    id: 1,
    name: "Laptop",
    price: 70000,
    stock: 5
  },
  {
    id: 2,
    name: "Mouse",
    price: 1000,
    stock: 0
  },
  {
    id: 3,
    name: "Keyboard",
    price: 2500,
    stock: 10
  }
];

function ShoppingCart() {
  const [cart, setCart] = useState([]);

  const addToCart = productId => {
    const product = initialProducts.find(
      product => product.id === productId
    );

    if (!product || product.stock === 0) return;

    setCart(prevCart => {
      const existingItem = prevCart.find(
        item => item.id === product.id
      );

      if (existingItem) {
        return prevCart.map(item =>
          item.id === product.id
            ? { ...item, quantity: item.quantity + 1 }
            : item
        );
      }

      return [
        ...prevCart,
        {
          ...product,
          quantity: 1
        }
      ];
    });
  };

  const totalPrice = cart.reduce(
```

```

    (total, item) => total + item.price * item.quantity,
    0
  );

  const hasOutOfStock = initialProducts.some(
    product => product.stock === 0
  );

  return (
    <div>
      <h2>Products</h2>

      {initialProducts.map(product => (
        <div key={product.id}>
          <h3>{product.name}</h3>
          <p>₹{product.price}</p>

          <button
            onClick={() => addToCart(product.id)}
            disabled={product.stock === 0}
          >
            {product.stock === 0 ? "Out of Stock" : "Add to Cart"}
          </button>
        </div>
      ))}

      <h2>Cart</h2>

      {cart.map(item => (
        <div key={item.id}>
          <p>
            {item.name} × {item.quantity}
          </p>
          <p>
            ₹{item.price * item.quantity}
          </p>
        </div>
      ))}

      <h2>Total: ₹{totalPrice}</h2>

      {hasOutOfStock && (
        <p>Some products are out of stock.</p>
      )}
    </div>
  );
}

export default ShoppingCart;

```

Methods Used

```
find()    → product বা cart item খোঁজা  
map()     → cart/product render এবং item update  
reduce()  → total price calculate  
some()    → out-of-stock check
```

Real-World React Project 3: Student Result Dashboard

এই project-এ `filter()`, `sort()`, `reduce()`, `map()` এবং `find()` ব্যবহার করা হয়েছে।

Features

```
Student list  
Pass/Fail filtering  
Marks অনুযায়ী sorting  
Total marks  
Student details
```

Code

```
import { useMemo, useState } from "react";

const students = [
  {
    id: 1,
    name: "Rahim",
    department: "CSE",
    marks: [80, 75, 90]
  },
  {
    id: 2,
    name: "Karim",
    department: "CSE",
    marks: [55, 60, 58]
  },
  {
    id: 3,
    name: "Hasan",
    department: "EEE",
    marks: [35, 45, 30]
  }
];

function StudentDashboard() {
  const [status, setStatus] = useState("all");
  const [selectedId, setSelectedId] = useState(null);

  const studentData = useMemo(() => {
    return students.map(student => {
      const total = student.marks.reduce(
        (sum, mark) => sum + mark,
        0
      );

      const average = total / student.marks.length;
      const passed = average >= 40;

      return {
        ...student,
        total,
        average,
        passed
      };
    });
  }, []);

  const filteredStudents = studentData
    .filter(student => {
      if (status === "passed") return student.passed;
      if (status === "failed") return !student.passed;
      return true;
    })
    .sort((a, b) => b.average - a.average);

  const selectedStudent = studentData.find(
    student => student.id === selectedId
  );
}
```

```

return (
  <div>
    <select
      value={status}
      onChange={e => setStatus(e.target.value)}
    >
      <option value="all">All Students</option>
      <option value="passed">Passed</option>
      <option value="failed">Failed</option>
    </select>

    <div>
      {filteredStudents.map(student => (
        <div key={student.id}>
          <h3>{student.name}</h3>
          <p>Department: {student.department}</p>
          <p>Total: {student.total}</p>
          <p>Average: {student.average.toFixed(2)}</p>

          <button
            onClick={() => setSelectedId(student.id)}
          >
            View Details
          </button>
        </div>
      ))}
    </div>

    {selectedStudent && (
      <div>
        <h2>{selectedStudent.name}</h2>
        <p>Marks: {selectedStudent.marks.join(", ")}</p>
        <p>Total: {selectedStudent.total}</p>
        <p>
          Status: {selectedStudent.passed ? "Passed" : "Failed"}
        </p>
      </div>
    )}
  </div>
);
}

export default StudentDashboard;

```

Methods Used

```

map()    → student data transform + UI render
filter() → passed/failed students filter
sort()   → average marks অনুযায়ী sort
reduce() → total marks calculate
find()   → selected student খুঁজে বের করা

```

React Array Methods: Quick Cheat Sheet

Method	Main Job	React Use Case
<code>map()</code>	Transform + return new array	List/Card/Table rendering
<code>filter()</code>	Matching items বের করা	Search/Category/Status
<code>find()</code>	First matching item	Single User/Product
<code>includes()</code>	Value আছে কিনা	Simple permission/category check
<code>some()</code>	At least one match	Validation/Warning/Condition
<code>reduce()</code>	One final result	Total/Count/Summary
<code>sort()</code>	Order change	Price/Name/Date/Score sorting

The Most Important Mental Model

```
API / State Data
  ↓
filter()
  ↓
sort()
  ↓
map()
  ↓
React UI
```

And when individual or summary data is needed:

```
find()      → one item
includes()   → value exists?
some()       → any item matches?
reduce()     → one final result
```

Important React Rule: Do Not Mutate State Directly

React state array-এর ক্ষেত্রে original array directly modify না করে নতুন array তৈরি করা উচিত।

| Avoid

```
products.push(newProduct);  
products.sort((a, b) => a.price - b.price);
```

| Prefer

```
setProducts(prev => [  
  ...prev,  
  newProduct  
]);
```

```
const sortedProducts = [...products].sort(  
  (a, b) => a.price - b.price  
);
```

For removing items:

```
setProducts(prev =>  
  prev.filter(product => product.id !== productId)  
);
```

For updating items:

```
setProducts(prev =>  
  prev.map(product =>  
    product.id === productId  
      ? { ...product, price: 50000 }  
      : product  
  )  
);
```

Priority for React Developers

Learn these in this order:

```
★★★★★ map()
★★★★★ filter()
★★★★★ find()
★★★★☆ includes()
★★★★☆ some()
★★★★☆ reduce()
★★★★☆ sort()
```

| Final Takeaway

For React development, master these seven:

```
map()
filter()
find()
includes()
some()
reduce()
sort()
```

The most important three are:

```
map()    → UI render
filter() → data filter
find()   → single data
```

Then add:

```
includes() → check value
some()     → check any match
reduce()   → calculate totals
sort()     → arrange data
```

Once these methods become comfortable, working with API responses, React state, product lists, dashboards, tables, search, filters, carts, and user data becomes much easier.